

Introduction to Machine Learning - 2

The Idea of Machine Learning

Machine Learning (ML) did not appear suddenly. It evolved slowly from deep theoretical ideas to practical systems that now power search engines, recommendation systems, self-driving cars, and AI assistants.

1936 – The Idea That Started It All: Alan Turing

In 1936, a 24-year-old genius young mathematician named Alan Turing introduced a revolutionary idea.

At a university in England, sitting in his room Turing was thinking

*“What if I could design a machine.....
that thinks?”*

This was the beginning of a revolution.

He created a theoretical model called the **Turing Machine**.

What Is a Turing Machine?

The Turing Machine is:

- A hypothetical (imaginary) machine
- Designed to follow a set of clear instructions (an algorithm)
- Capable of solving any problem that can be expressed step-by-step

Simple Explanation

Imagine:

- A long tape divided into small boxes (like memory)
- A head that can read and write symbols on the tape
- A list of rules telling the machine what to do

If you give it:

- Clear rules
- Enough time
- Enough tape

It can simulate any algorithm.

Why This Was Important

- It defined what “computation” means.
- It proved that machines could, in theory, perform logical thinking.
- It became the foundation of modern computers.

Without this concept, there would be no programming, no algorithms, and no machine learning.

1950 – “Can Machines Think?”

In 1950, Turing wrote a famous paper titled:

“Computing Machinery and Intelligence”

In it, he asked a bold question:

Can machines think?

Instead of arguing philosophically, he proposed a practical experiment called the **Turing Test**.

The Turing Test

The idea:

- A human judge chats with two hidden participants:
 - One human
 - One machine
- If the judge cannot reliably tell which is the machine, then the machine can be considered intelligent.

Why This Matters

This was the beginning of Artificial Intelligence (AI) as a field.

It shifted the question from:

- “Do machines have minds?”

to:

- “Can machines behave intelligently?”

This shift laid the foundation for Machine Learning.

1957 – The First Learning Algorithm: The Perceptron

In 1957, psychologist Frank Rosenblatt created the **Perceptron(First Algorithm that Learned from Data)**.

This was one of the first machine learning algorithms.

What Is a Perceptron?

The Perceptron is:

- A simple model inspired by the human brain
- A basic artificial neuron
- Able to learn from data

How It Works

Imagine you want to classify emails as:

- Spam
- Not spam

The perceptron:

1. Looks at features (like certain words).
2. Assigns weights (importance) to them.
3. Adjusts those weights when it makes mistakes.
4. Gradually improves.

This was revolutionary because:

- The machine was not just following fixed rules.
- It was adjusting itself based on data.

Limitations

However:

- It could only solve simple linear problems.
- It failed at more complex tasks.
- Researchers criticized it heavily.

Funding dropped. AI entered a period called the “AI Winter.”

But the idea of learning from data survived.

1980s – Neural Networks Return

In the 1980s, researchers revived neural networks.

A major breakthrough came in 1986 with the rediscovery and popularization of **backpropagation** (The foundational, iterative algorithm for training artificial neural networks using gradient descent).

What Changed?

Earlier perceptron had limits.

Now researchers:

- Stacked multiple layers of neurons
- Developed algorithms to adjust weights efficiently
- Enabled networks to learn complex patterns

What Could They Do?

- Recognize handwritten digits
- Detect patterns in data
- Make predictions

These systems were still small compared to modern AI, but they proved something important:

Machines could learn complex relationships from data.

This period quietly restarted Machine Learning research.

1997 – A Historic Moment: Deep Blue

In 1997, IBM's chess computer defeated world chess champion Garry Kasparov.

The computer was called Deep Blue, built by IBM.

Why This Was Huge

- Chess was considered a symbol of human intelligence.
- Kasparov was the best player in the world.
- A machine beat him in a match.

Although Deep Blue relied heavily on:

- Search algorithms
- Evaluation functions

- Massive computation power

It showed that machines could outperform humans in complex intellectual tasks.

This event changed public perception:

Machines were no longer “just calculators.”

2016 – AlphaGo

In 2016, a computer program defeated one of the world’s greatest Go players, Lee Sedol.

The program was called AlphaGo, developed by DeepMind (later acquired by Google).

This was not just another game victory.

It was something much bigger.

Why This Was Huge

Go is very different from chess.

- Go has far more possible board positions than chess.
- The number of possible moves is astronomically large.
- Brute-force search (like in chess engines) is not enough.
- It requires deep intuition and pattern recognition.

Many experts believed it would take decades before a machine could defeat a top human Go player.

Yet AlphaGo won the match against Lee Sedol 4–1.

That shocked the world.

Why Go Was So Hard for Machines

To understand the importance, compare it with chess:

- In chess, computers can calculate many moves ahead.
- In Go, the number of possible positions is larger than the number of atoms in the observable universe.
- Human Go players rely heavily on intuition, not just calculation.

This meant traditional search-based AI (like Deep Blue) was not enough.

A different approach was required.

How AlphaGo Worked

Unlike Deep Blue, AlphaGo did not rely only on brute force.

It combined:

- Deep Neural Networks
- Reinforcement Learning
- Monte Carlo Tree Search

1. Neural Networks

AlphaGo used deep neural networks to:

- Predict the best next move (Policy Network)
- Estimate who is winning from a board position (Value Network)

Instead of calculating everything, it learned patterns from data.

2. Reinforcement Learning

After learning from human games, AlphaGo improved by:

- Playing millions of games against itself
- Learning from wins and losses
- Adjusting its strategy over time

This is called self-play reinforcement learning.

3. Smart Search (Not Blind Search)

It did not explore all possibilities.

Instead, it:

- Focused on promising moves
- Ignored unlikely paths
- Combined intuition (neural networks) with strategic exploration

This was a new era of AI.

Deep Blue calculated.

AlphaGo learned.

Why This Moment Changed Everything

When Lee Sedol lost, it was emotional.

It wasn't just about a game.

It showed that machines were no longer limited to:

- Logical calculation
- Structured problems

They could now:

- Recognize complex patterns
- Develop creative strategies
- Discover moves humans had never considered

One of AlphaGo's moves in Game 2 (Move 37) was so unusual that experts first thought it was a mistake. Later, they realized it was brilliant.

That moment symbolized something powerful:

Machines were not just copying humans.

They were innovating.

Move 37 – A “Creative” Moment

During the match against Lee Sedol, AlphaGo made a move that stunned the entire Go world.

It happened in **Game 2** of the 2016 match.

The move is now famously known as **Move 37**.

Move 37 was later analyzed and praised as:

- Deep
- Flexible
- Highly creative

It wasn't random.

It came from patterns learned through millions of games and self-play reinforcement learning.

Geoffrey Hinton – The Godfather of Deep Learning

- Geoffrey Hinton is a British-Canadian computer scientist and cognitive psychologist.
- He is widely known as one of the foundational figures in modern **machine learning and deep learning**.

Many people call him the “**Godfather of AI**” because his ideas helped shape the way intelligent systems are designed and trained today

AlexNet – A Milestone in Deep Learning

- AlexNet is a **deep convolutional neural network (CNN)** that dramatically advanced image recognition.
- Developed by **Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton** in 2012.
- It won the **ImageNet Large Scale Visual Recognition Challenge (ILSVRC)** with a huge margin, reducing error rates significantly.

Generative Adversarial Networks (GANs) – Creating AI That Imagines

- A **Generative Adversarial Network (GAN)** is a type of neural network that can **generate new data** (like images, music, or text) that resembles real data.
- Introduced in **2014** by **Ian Goodfellow** and his collaborators.
- GANs are famous for creating highly realistic images, deepfakes, and artistic AI content.

How GANs Work (Simplified)

A GAN consists of **two neural networks** competing with each other:

1. Generator

- Creates fake data from random noise.
- Tries to make the output look as realistic as possible.

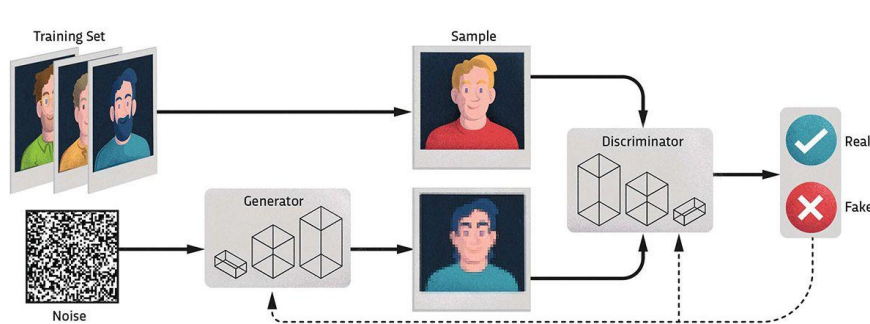
2. Discriminator

- Evaluates data and tries to **distinguish real from fake**.
- Sends feedback to the generator.

The “Adversarial” Game

- The generator wants to **fool the discriminator**.
- The discriminator wants to **catch the fakes**.
- Over time, both improve:
 - Generator becomes better at creating realistic data.
 - Discriminator becomes better at spotting fakes.

Think of it like a counterfeiter and a police officer improving each other’s skills.



Sequence-to-Sequence (Seq2Seq) Networks – Teaching Machines to Translate Sequences

- A **Sequence-to-Sequence (Seq2Seq) network** is a type of neural network designed to **transform one sequence into another**.
- Introduced around **2014** by researchers including Ilya Sutskever, Oriol Vinyals, and Quoc Le.
- Used widely in **machine translation, speech recognition, and text summarization**.

How Seq2Seq Networks Work

Seq2Seq models are usually built with **two main parts**:

1. **Encoder**
 - Reads the input sequence (e.g., a sentence in English).
 - Compresses the sequence into a **context vector** (a summary of its meaning).

2. Decoder

- Takes the context vector and generates the output sequence (e.g., the sentence in French).
- Produces one element at a time, using previous outputs as input.

Think of it like **reading a sentence, understanding it, and rewriting it in another language.**

Year	Innovation	Key Contribution
2015	ResNet	Deep residual learning for very deep neural networks
2017	Transformer	Attention-based sequence modeling for NLP
2020	GPT-3	Massive pre-trained language model for text generation
2023	Bard	Conversational AI chatbot with web access, powered by Language Model for Dialogue Applications (LaMDA)

Types of Learnings

1. Supervised Learning

Imagine you are back in **first grade**:

- You sit at your desk.
- Your teacher shows you a flashcard with a picture of an animal.
- She says: *“This is a cat.”*
- Next card: *“This is a dog.”*

Over time, your brain starts recognizing patterns:

- Cats usually have pointy ears.
- Dogs often have longer snouts.

You didn’t memorize every animal explicitly — you **learned from examples**.

This is exactly how **supervised learning** works in machines.

Why It's Called "Supervised"

- There is a **teacher or supervisor**.
- The machine is given **input data** (like pictures) and the **correct output** (labels like "cat" or "dog").
- The machine's job: **learn the relationship** between input and output.

Think of it as:

Teacher shows → Machine observes → Machine learns → Machine predicts

Real-World Example: Spam Email Detection

1. You have a set of emails. Some are spam, some are not.
2. Each email is **labeled**: spam or not spam.
3. The machine looks at patterns in the emails:
 - Words like "Congratulations!"
 - Multiple exclamation marks
 - Requests for money
4. Once trained, the machine can **predict** whether new emails are spam, even if it hasn't seen them before.

This is supervised learning in action: **learning from labeled examples to make predictions on new data.**

Common Supervised Learning Algorithms

Some widely-used algorithms include:

- **Linear Regression** – Predicts numbers (e.g., house prices).
- **Logistic Regression** – Predicts categories (e.g., yes/no, spam/not spam).
- **Decision Trees** – Makes decisions like a flowchart.
- **Support Vector Machines (SVM)** – Finds the best boundary between categories.

All these methods **learn from labeled data** and **predict outputs for new inputs**.

Why Supervised Learning Is Popular

- **Labeled data is everywhere:**
 - Doctors label X-rays: “cancer” or “healthy.”
 - Users label product reviews: “positive” or “negative.”
 - Banks label loan applications: “defaulted” or “paid back.”
- The more labeled examples a model sees, the better it becomes at prediction.

Limitations

- **Requires labeled data:** If no one labels the data (no answers provided), supervised learning cannot work.
- In situations where labels are unavailable, other methods like **unsupervised learning** or **reinforcement learning** are needed.

2. Unsupervised Learning

Imagine you have **hundreds of photos** from vacations, family gatherings, and random events:

- None of the photos are labeled.
- You don't tell the machine what is in each photo.

You ask a system to **sort them automatically**:

1. The machine analyzes **colors, shapes, faces, and scenery**.
2. It starts grouping photos based on similarities.

The resulting clusters might look like this:

- Beach photos together
- Family portraits together
- Food pictures together
- City sightseeing together

No one told the machine what a “beach” or “family” photo is.
It discovered patterns **on its own**.

What Is Unsupervised Learning?

- Machines **learn patterns without answers**.
- There is **no teacher** and **no labels**.
- You give the machine a pile of data and say:

“Go figure it out.”

- The machine finds patterns, groups similar items, or compresses information.

Think of it as **organizing chaos without knowing the outcome**.

Why “Unsupervised”?

- Simple: **no supervision**.
- Machines are left to explore the data on their own.
- They use **mathematics and statistics** to identify similarities and patterns.

Example: organizing random photos without knowing who or what is in them.

Real-World Example: Customer Segmentation

Suppose you run an online store:

- You have thousands of customers.
- You know some behaviors:
 - What they buy
 - How often they visit
 - How much they spend

Using unsupervised learning, the system might automatically group customers as:

- **Bargain hunters**
- **Big spenders**
- **Night-time browsers**

No one labeled these groups — the machine discovered them.
This lets you target marketing and offers more effectively.

Common Algorithms

You don't need to memorize these, but here are widely-used unsupervised learning methods:

- **K-means Clustering** – Groups similar items into clusters.
- **Hierarchical Clustering** – Builds a “family tree” of data relationships.
- **Principal Component Analysis (PCA)** – Reduces high-dimensional data into simpler form while keeping important patterns.

“I don't know what this is. But I can find what's similar.”

When Do We Use Unsupervised Learning?

- When there is **no labeled data**.
- When we want to **discover hidden patterns**.
- When we want to **explore data before making decisions**.

It may not always be perfectly accurate, but it's **extremely useful** for exploration and insight.

3. Reinforcement Learning - Learning by Doing

- Learning by **trial and error**.
- Learning from **rewards and punishments**.
- Machines learn **strategies** rather than memorizing answers.

Like humans or animals:

- Kids learn not to touch a hot pan.
- Dogs sit to get treats.

Machines can do the same — only faster and at scale.

Why “Reinforcement”?

- Every action gets **feedback** (positive or negative).
- This feedback is called a **reward**.
- The agent's goal: **figure out the sequence of actions that maximizes total reward over time**.

Real-World Analogy: Playing a Video Game

- Imagine playing **Super Mario**:
 - You fall into holes → punishment.
 - You bump into enemies → punishment.
 - You figure out when to jump, when to run → rewards.
- You learn by **trying, failing, and improving**.
- Reinforcement learning allows machines to do this **thousands of times faster**, without getting bored.

Key Terms

Term	Meaning
Agent	The learner (AI, or bot)
Environment	The world it interacts with (game, road)
Action	What the agent does at each step
Reward	Score or feedback after an action (good or bad)
Policy	The strategy the agent develops over time
Goal	Maximize total reward

The AlphaGo Story (2015–2016)

- Google DeepMind built **AlphaGo**, a reinforcement learning system for the game of **Go**.
- Steps it took:
 1. Learned from human games.
 2. Played **millions of games against itself** (self-play).
 3. Learned from every win and loss.
- Result: In **2016**, AlphaGo defeated world champion **Lee Sedol** — a historic moment in AI.

Reinforcement learning allowed AlphaGo to **learn strategies humans never discovered**.

Real-World Applications of Reinforcement Learning

1. **Self-driving cars** – Learn to drive safely in simulations before going on the road.
2. **Robotics** – Teach robots to walk, pick objects, or open doors.
3. **Finance** – Learn optimal buying/selling strategies based on reward signals.
4. **Game AI** – From chess (Deep Blue) to Go (AlphaGo) to modern eSports games like Dota and StarCraft.

Reinforcement learning is **learning by interaction**:

- Explore → Act → Receive feedback → Improve
- Machines develop **strategies over time**, not just memorize rules.
- This type of learning powers many **cutting-edge AI systems** today, from game champions to robots and self-driving cars.

Example: Predicting if Someone Likes Basketball

Scenario:

- We have data about a few people: their **age** and **hours of weekly exercise**.
- We want to predict if they **like basketball** or not.
- Instead of writing “if-else” rules, we’ll train a model to learn patterns.

Step 1: Install Scikit-learn

```
!pip install scikit-learn
```

Step 2: Collect Data

```
from sklearn.tree import DecisionTreeClassifier
```

```
# Features: [age (years), exercise hours per week]
```

```
features = [
```

```
    [10, 5], # young and active
```

```
    [12, 3], # young and moderately active
```

```
    [25, 1], # adult, low activity
```

```
[30, 0], # adult, no activity  
]  
  
# Labels: "likes_basketball" or "does_not_like"  
labels = ["yes", "yes", "no", "no"]
```

Step 3: Train the Model

```
clf = tree.DecisionTreeClassifier()  
  
clf = clf.fit(features, labels)
```

Step 4: Make Predictions

```
# New person: 11 years old, exercises 4 hours/week  
print(clf.predict([[11, 4]])) # Output: yes  
  
# New person: 28 years old, exercises 1 hour/week  
print(clf.predict([[28, 1]])) # Output: no
```

What Is a Classifier?

A **classifier** is a type of **machine learning model** that helps a computer **categorize or label data**.

- It **takes input data** (for example, the age, practice hours).
- It **learns from examples** — this is called **training**.
- It **predicts labels** for new, unseen data (like saying “like” or “do not like”).

This is the essence of **supervised learning**: learning from labeled examples to make predictions.

Features and Labels

In machine learning, we work with two main components:

1. **Features** – Characteristics or attributes of the data.

Examples:

- Age
- Color
- Texture

- Number of hours

2. **Labels** – The categories or outcomes we want to predict.

Examples:

- “Apple” or “Orange”
- “Spam” or “Not Spam”
- “Likes Basketball” or “Does Not Like”

